Programare funcțională

Introducere în programarea funcțională folosind Haskell

C02 - Tipuri de date

Denisa Diaconescu

Ana Iova

Departamentul de Informatică, FMI, UB

Tipuri de date

Tipuri de date. Sistemul tipurilor

"There are three interesting aspects to types in Haskell: they are strong, they are static, and they can be automatically inferred."

http://book.realworldhaskell.org/read/types-and-functions.html

strong – garantează absența anumitor erori

static – tipul fiecărei valori este calculat la compilare

dedus automat – compilatorul deduce automat tipul fiecărei expresii

```
ghci> :t [('a',1,"abc")]
[('a',1,"abc")] :: Num b => [(Char, b, [Char])]
```

Sistemul tipurilor

Tipurile de bază: Int, Integer, Float, Double, Bool, Char, String

Tipuri compuse: tupluri și liste

```
ghci> :t ('a', True)
('a', True) :: (Char, Bool)

ghci> :t ["Tom", "Jerry"]
["Tom", "Jerry"] :: [[Char]]
```

Tipuri noi definite:

```
data RGB = Red | Green | Blue
data Point a = Pt a a     -- tip parametrizat
                          -- a este variabila de tip
```

Tipuri de date

Integer: 4, 0, -5

```
ghci> 4 + 3
ghci> (+) 4 3
```

```
ghci> mod 4 3
ghci>  4 `mod` 3
```

Float: 3.14

```
ghci> truncate 3.14
ghci> sqrt 4
```

```
ghci> let x = 4 :: Int
ghci> sqrt (fromIntegral x)
```

Char: ’a’,’A’, ’\n’

```
ghci> import Data.Char
Prelude Data.Char> chr 65
Prelude Data.Char> ord 'A'
Prelude Data.Char> toUpper 'a'
Prelude Data.Char> digitToInt '4'
```

Tipuri de date

Bool: True, False

```
data Bool = True | False
```

```
ghci> True && False || True
ghci> not True
```

```
ghci> 1 /= 2
ghci> 1 == 2
```

String: "Tom and Jerry"

```
type String = [Char] -- sinonim pentru tip
```

```
ghci> "Tom"++"Jerry"
"TomJerry"
ghci> "TomJerry" !! 2
'm'
```

```
ghci> lines "Tom\nJerry"
["Tom","Jerry"]
ghci> words "Tom␣and␣Jerry"
["Tom","and","Jerry"]
```

Tipuri de date compuse

Tipul listă

```
ghci>:t [True, False, True]
[True, False, True] :: [Bool]
```

Tipul tuplu

```
ghci> :t (1 :: Int, 'a', "ab")
(1 :: Int, 'a', "ab") :: (Int, Char, [Char])

ghci> fst (1,'a') -- doar pentru perechi
ghci> snd (1,'a') -- doar pentru perechi
```

Tipuri. Clase de tipuri. Variabile de tip

Ce răspuns primim?

```
ghci> :t 1
```

Răspunsul este:

```
1 :: Num a => a
```

Semnificația este următoarea:

- Num este o clasă de tipuri
- a este un parametru de tip
- 1 este o valoare de tipul a din clasa Num

```
ghci> :t [1,2,3]
[1,2,3] :: Num t => [t]
```

Erori de tip

Eroarea pe care o să o întâlniți cel mai des:

Couldn’t match expected type

```
ghci> "string" ++ True
<interactive>:1:13: error: [GHC-83865]
    * Couldn't match expected type `[Char]' with actual type `Bool'
    * In the second argument of `(++)', namely `True'
      In the expression: "string" ++ True
      In an equation for `it': it = "string" ++ True
```

Chiar dacă pare lungă și scary, lucrurile nu stau chiar așa.

Don’t panic: citiți eroarea.

Funcții

Funcții

Prototipul funcției

double :: Integer -> Integer

- **numele funcției**
- signatura funcției

Definiția funcției

double elem = elem + elem

- **numele funcției**
- parametrul formal
- corpul funcției

Aplicarea funcției

double 5

- **numele funcției**
- parametrul actual (argumentul)

Exemplu: funcție cu două argumente

Prototipul funcției **add** :: Integer -> Integer -> Integer

- **numele funcției**
- signatura funcției

Definiția funcției **add** elem1 elem2 = elem1 + elem2

- **numele funcției**
- parametrii formali
- corpul funcției

Aplicarea funcției **add** 3 7

- **numele funcției**
- argumentele

Exemplu: funcție cu un argument de tip tuplu

Prototipul funcției **dist** :: (Integer, Integer) -> Integer

- **numele funcției**
- signatura funcției

Definiția funcției **dist** (elem1, elem2) = abs (elem1 - elem2)

- **numele funcției**
- parametrul formal
- corpul funcției

Aplicarea funcției **dist** (5, 7)

- **numele funcției**
- argumentul

Exemple de tipuri de funcții

```
ghci> :t abs
abs :: Num a => a -> a

ghci> :t (:)
(:) :: a -> [a] -> [a]

ghci> :t (++)
(++) :: [a] -> [a] -> [a]

ghci> :t zip
zip :: [a] -> [b] -> [(a, b)]
```

Definirea funcțiilor - Recursivitate

- O funcție este recursivă dacă se apelează pe ea însăși.
- Pentru ca o definiție recursivă să se termine, sunt necesare:
  - unul sau mai multe cazuri de bază;
  - unul sau mai multe apeluri recursive care apropie problema de cazul de bază.

Exemplu: Suma numerelor naturale până la n

```
sumN :: Integer -> Integer
sumN n = if n == 0 then 0 else n + sumN (n - 1)
```

```
sumN 4
    = 4 + sumN 3
    = 4 + 3 + sumN 2
    = 4 + 3 + 2 + sumN 1
    = 4 + 3 + 2 + 1 + sumN 0
    = 4 + 3 + 2 + 1 + 0
    = 10
```

Definirea funcțiilor

```
fact :: Integer -> Integer
```

- Definiție folosind **if**

```
fact n = if n == 0 then 1
         else n * fact(n-1)
```

- Definiție folosind pattern matching

```
fact 0 = 1
fact n = n * fact(n-1)
```

- Definiție folosind guards

```
fact n
  | n == 0    = 1
  | otherwise = n * fact(n-1)
```

Pattern matching

```
fact :: Integer -> Integer
fact 0 = 1
fact n = n * fact(n-1)
```

- variabilele și valorile din partea stângă a lui = sunt șabloane (*patterns*)
- când funcția este apelată se încearcă potrivirea parametrilor actuali, ecuațiile fiind încercate în ordinea scrierii
- în definiția factorialului, 0 și n sunt patterns:
  0 se va potrivi numai cu el însuși,
  iar n se va potrivi cu orice valoare de tip `Integer`

Pattern matching

Ordinea ecuațiilor este importantă!

Să presupunem că schimbăm ordinea ecuațiilor din definiția factorialului:

```
fact :: Integer -> Integer
fact n = n * fact(n-1)
fact 0 = 1
```

Ce se întâmplă?

Deoarece n este un pattern care se potrivește cu orice valoare, inclusiv cu 0, orice apel al funcției va alege prima ecuație. Astfel, funcția nu își va încheia execuția.

Pattern matching

Tipul Bool este definit în Haskell astfel:

```
data Bool = True | False
```

Putem defini operația || astfel

```
(||) :: Bool -> Bool -> Bool
```

```
False || x = x
True  || _ = True
```

În acest exemplu _, **x**, **True** și **False** sunt patterns.

Observăm că **True** și **False** sunt constructori de date și se vor potrivi numai cu ei înșiși.

Pattern-ul _ se numește *wild-card pattern*; el se potrivește cu orice valoare.

Tipuri de funcții

Fie foo o funcție cu următorul tip

```
foo :: a -> b -> [a] -> [b]
```

- are trei argumente, de tipuri a, b și [a]
- întoarce un rezultat de tip [b]

Schimbăm signatura funcției astfel:

```
ffoo :: (a -> b) -> [a] -> [b]
```

- are două argumente, de tipuri (a -> b) și [a],
  adică o funcție de la a la b și o listă de elemente de tip a
- întoarce un rezultat de tip [b]

```
ghci> :t map
map :: (a -> b) -> [a] -> [b]
```

Quiz time!



https://tinyurl.com/PF-C02-Quiz1

Liste

Liste

Orice listă poate fi scrisă folosind doar constructorul (:) și lista vidă [].

- [1,2,3] == 1 : (2 : (3 : [])) == 1 : 2 : 3 : []
- "abc" == ['a','b','c'] == 'a' : ('b' : ('c' : [])) == 'a' : 'b' : 'c' : []

Definiție recursivă. O **listă** este

- vidă, notată [ ], sau
- compusă, notată x : xs, dintr-un element x numit **capul listei** *(head)* și o listă xs numită **coada listei** *(tail)*.

Definirea listelor. Operații

Intervale și progresii

```
interval = ['c'..'e']         -- ['c','d','e']
progresie = [20,17..1]        -- [20,17,14,11,8,5,2]
progresie' = [2.0,2.5..4.0]   -- [2.0,2.5,3.0,3.5,4.0]
```

Operații

```
ghci> import Data.List
ghci> [1,2,3] !! 2
3
ghci> "abcd" !! 0
'a'
ghci> [1,2] ++ [3]
[1,2,3]
```

Definiția prin selecție $\{x \mid P(x)\}$

$$[\ E(x) \mid x <\!\!- [x_1, \ldots, x_n],\ P(x)\ ]$$

```
ghci> xs = [0..10]
ghci> [x | x <- xs, even x]
[0,2,4,6,8,10]

ghci> xs = [0..6]
ghci> [(x,y) | x <- xs, y <- xs,  x + y == 10]
[(4,6),(5,5),(6,4)]
```

Definiția prin selecție $\{x \mid P(x)\}$

$$[\ E(x) \mid x <\!\!- [x_1, \ldots, x_n],\ P(x)\ ]$$

Putem folosi `let` pentru domeniu de vizibilitate local.

```
ghci> [(i,j) | i <- [1..2],
                  let k = 2 * i, j <- [1..k]]
[(1,1),(1,2),
        (2,1),(2,2),(2,3),(2,4)]
```

zip xs ys

```
ghci> :t zip
zip :: [a] -> [b] -> [(a, b)]

ghci> ys = ['A'..'E']
ghci> zip [1..]  ys
[(1,'A'),(2,'B'),(3,'C'),(4,'D'),(5,'E')]

ghci> xs = ['A'..'Z']
ghci> [x | (i,x) <- [1..] `zip` xs, even i]
"BDFHJLNPRTVXZ"
```

zip xs ys

Observați diferența!

```
ghci> zip [1..3] ['A'..'D']
[(1,'A'),(2,'B'),(3,'C')]

ghci> [(x,y) | x <- [1..3], y <- ['A'..'D']]
[(1,'A'),(1,'B'),(1,'C'),(1,'D'),
        (2,'A'),(2,'B'),(2,'C'),(2,'D'),
                (3,'A'),(3,'B'),(3,'C'),(3,'D')]
```

Lazy

Lazy: argumentele sunt evaluate doar când e necesar și doar cât e necesar.

```
ghci> head[]
*** Exception: Prelude.head: empty list
ghci> x = head []
ghci> f a = 5
ghci> f x
5
ghci> [1,head [],3] !! 0
1
ghci> [1, head [],3] !! 1
*** Exception: Prelude.head: empty list
```

Liste infinite

Drept consecință a evaluării lazy, se pot defini liste infinite.

```
ghci> natural = [0..]
ghci> take 5 natural
[0,1,2,3,4]

ghci> evenNat = [0,2..] -- progresie infinita
ghci> take 7 evenNat
[0,2,4,6,8,10,12]

ghci> ones = [1,1..]
ghci> zeros = [0,0..]
ghci> both = zip ones zeros
ghci> take 5 both
[(1,0),(1,0),(1,0),(1,0),(1,0)]
```

Șabloane (patterns) pentru liste

Listele sunt construite folosind constructorii (:) și []

```
[1,2,3] == 1:[2,3]  == 1:2:[3] == 1:2:3:[]
```

```
ghci> x:y = [1,2,3]
ghci> x
1
ghci> y
[2,3]
```

Ce s-a întâmplat?

- x : y este un pattern pentru liste
- potrivirea dintre x : y și [1,2,3] a avut ca efect:
  - "deconstrucția" valorii [1,2,3] în 1 : [2,3]
  - legarea lui x la 1 și a lui y la [2,3]

Șabloane (patterns) pentru liste

Definiții folosind șabloane

```
reverse [] = []
reverse (x:xs) = (reverse xs) ++ [x]
```

x : xs se potrivește cu liste nevide.

Șabloanele sunt definite folosind constructori.

De exemplu, operația de concatenare pe liste este

```
(++) ::  [a] -> [a] -> [a],
```

dar `[x] ++ [1] = [2,1]` nu va avea ca efect legarea lui x la 2.

```
ghci> [x] ++ [1] = [2,1]
ghci> x
error:  ...
```

Șabloanele sunt liniare

Șabloanele sunt liniare, adică o variabilă apare cel mult dată.

Șabloanele în care o variabilă apare de mai multe ori generează mesaje de eroare. De exemplu:

```
- x:x:[1] = [2,2,1]
- ttail (x:x:t) = t
- foo x x = x^2
error:  Conflicting definitions for x
```

O soluție este folosirea gărzilor:

```
ttail (x:y:t) | (x==y) = t
              | otherwise = ...

foo x y | (x == y) = x^2
        | otherwise = ...
```

Operatori

Operatorii sunt funcții cu două argumente

Operatorii în Haskell

- au două argumente
- pot fi apelați folosind notația infix
- pot fi definiți folosind numai "simboluri" (ex: *!*)
  - în definiția tipului operatorul este scris între paranteze
- Operatori predefiniți

```
(||) :: Bool -> Bool -> Bool
 (:) :: a -> [a] -> [a]
 (+) :: Num a => a -> a -> a
```

- Operatori noi

```
(&&&) :: Bool -> Bool -> Bool  -- atentie la paranteze
True &&& b = b
False &&& _ = False
```

Funcții ca operatori

Operatorii care sunt definiți în formă infix, sunt apelați în formă prefix folosind paranteze

```
2 + 3 == (+) 2 3
```

Operatorii care sunt definiți în formă prefix, sunt apelați în formă infix folosind `` ` `` *(backtick)*

```
mod 5 2 == 5 `mod` 2
```

```
ghci> mod 5 2
1
ghci> 5 `mod` 2
1
```

```
elem :: a -> [a] -> Bool
ghci> 1 `elem` [1,2,3]
True
```

Precedență și asociativitate

```
ghci> 3+5*4:[6]++8-2+3:[2]==[23,6,9,2]||True==False
True
```

Precedence	Left associative	Non-associative	Right associative
9	`!!`		`.`
8			`^`, `^^`, `**`
7	`*`, `/`, `` `div` ``, `` `mod` ``, `` `rem` ``, `` `quot` ``		
6	`+`, `-`		
5			`:`, `++`
4		`==`, `/=`, `<`, `<=`, `>`, `>=`, `` `elem` ``, `` `notElem` ``	
3			`&&`
2			`||`
1	`>>`, `>>=`		
0			`$`, `$!`, `` `seq` ``

Asociativitate

Operatorul - asociativ la stânga

```
5 - 2 - 1 == (5 - 2)- 1                    -- /= 5 - (2 - 1)
```

Operatorul : asociativ la dreapta

```
5 : 2 : [] == 5 : (2 : [])
```

Operatorul ++ asociativ la dreapta

```
(++) :: [a] -> [a] -> [a]
[] ++ ys = ys
(x:xs) ++ ys = x:(xs ++ ys)

l1 ++ l2 ++ l3 ++ l4 ++ l5 == l1 ++ (l2 ++ (l3 ++ (l4 ++
 l5)))
```

Quiz time!



https://tinyurl.com/PF-C02-Quiz2

Pe săptămâna viitoare!